

Configuração do *ambiente*

Antes do primeiro prompt, cinco peças precisam estar no lugar: a **chave da API** do Groq, uma **pasta de projeto**, um **ambiente virtual** (venv), as **bibliotecas** do `requirements.txt` e o arquivo `.env` com a chave. No fim, um teste rápido mostra se tudo conversa. O passo a passo detalhado para Windows, Linux e macOS está no manual A4 desta aula.

Chave da API

Pasta Eng_prompt

venv

requirements.txt

.env

Teste

Formação Claude Code 2026 · Baseado em console.groq.com/docs/quickstart, docs.python.org/3/library/venv.html, jupyterlab.readthedocs.io e pypi.org, consultados em outubro de 2026 · Testado com groq 1.7.0 e python-dotenv 1.2.4; mensagens de erro reais em [exemplos/](#)

O que vamos *montar*

Pasta do projeto ao final

```
Eng_prompt/
├── .venv/           # ambiente virtual (não vai para o Git)
├── .env             # GROQ_API=sua_chave (secreto!)
├── .gitignore       # ignora .env e .venv
├── requirements.txt # python-dotenv, jupyterlab, groq
├── teste_groq.py    # teste da conexão
└── 01-roles.ipynb ... # notebooks das aulas 04 em diante
```

- ✓ **Python 3.10 ou mais novo:** é o mínimo exigido pelas versões atuais de `groq`, `jupyterlab` e `python-dotenv`
- ✓ O `requirements.txt` está na pasta `Eng_prompt/` da aula 01
- ✓ Os arquivos `teste_groq.py`, `.env.exemplo` e `.gitignore.exemplo` estão em `exemplos/` nesta aula

Comandos de terminal mudam entre sistemas. Aqui está a versão Linux/macOS, com a de Windows ao lado quando for diferente; o manual traz os três sistemas completos.

Crie a sua *chave da API*

1

Entre no console

Acesse `console.groq.com` e crie uma conta (ou entre com a sua).

2

Abra API Keys

Página `console.groq.com/keys`.

3

Create API Key

Dê um nome (ex.: "curso-prompts") e copie a chave gerada.

4

Guarde num lugar seguro

Ela vai para o arquivo `.env` no passo 4.

A chave é uma senha. Quem a tiver usa a sua cota. Não cole em código, prints, e-mails ou repositórios públicos. Se vazar, apague-a no console e crie outra.

A documentação do Groq recomenda guardar a chave numa **variável de ambiente**, em vez de escrevê-la no código. É o que o `.env` faz.

Pasta e *ambiente virtual*

Linux e macOS

```
mkdir Eng_prompt && cd Eng_prompt
python3 -m venv .venv
source .venv/bin/activate
```

Windows (PowerShell)

```
mkdir Eng_prompt; cd Eng_prompt
py -m venv .venv
.venv\Scripts\Activate.ps1
```

- ✓ O venv é uma instalação de Python **só deste projeto**: as bibliotecas não se misturam com as de outros projetos
- ✓ Ativado, o terminal mostra `(.venv)` no começo da linha
- ✓ Ative de novo sempre que abrir um terminal novo

No Windows, se o PowerShell bloquear o `Activate.ps1`, rode uma vez:

```
Set-ExecutionPolicy -ExecutionPolicy RemoteSigned -Scope
CurrentUser
```

Instale as *bibliotecas*

Eng_prompt/requirements.txt (da aula 01)

```
python-dotenv
jupyterlab
groq
```

Com o venv ativado

```
pip install -r requirements.txt
```

BIBLIOTECA	PARA QUÊ
groq	SDK oficial: o <code>Groq()</code> e o <code>client.chat.completions.create</code>
python-dotenv	<code>load_dotenv()</code> : lê o <code>.env</code> e põe a chave no ambiente
jupyterlab	A interface dos notebooks (aula 03)

Sem versões fixas no arquivo, o pip instala as mais recentes. Em outubro de 2026: groq 1.7.0, jupyterlab 4.6.4, python-dotenv 1.2.4.

O arquivo *.env*

Eng_prompt/.env

```
GROQ_API=cole_aqui_a_sua_chave
```

Eng_prompt/.gitignore

```
# Nunca suba a chave da API para o Git
.env
.venv/
.ipynb_checkpoints/
```

- ✓ Uma linha, sem aspas e sem espaços em volta do `=`
- ✓ O nome do arquivo é só `.env` (começa com ponto, sem extensão)
- ✓ `load_dotenv()` procura o `.env` a partir da pasta onde o código roda

Atenção ao nome: os notebooks usam `GROQ_API`. O SDK, sozinho, procura `GROQ_API_KEY`. Por isso o código passa `api_key=os.environ.get("GROQ_API")`; sem isso, a chave não é encontrada.

Teste *a conexão*

exemplos/teste_groq.py

```
from dotenv import load_dotenv
from groq import Groq
import os

load_dotenv() # lê o arquivo .env da pasta atual
chave = os.environ.get("GROQ_API")
print("Chave carregada?", "sim" if chave else "não")

client = Groq(api_key=chave)
resposta = client.chat.completions.create(
    model="openai/gpt-oss-120b",
    messages=[{"role": "user", "content": "Diga olá em uma
frase."}],
)
print(resposta.choices[0].message.content)
```

Rodando

```
python teste_groq.py
```

- ✓ Deu certo: aparece `Chave carregada? sim` e, em seguida, uma frase de saudação do modelo
- ✓ É a mesma estrutura da primeira célula de todos os notebooks
- ✓ Se aparecer erro, veja a próxima seção

Os dois erros *mais comuns*

SEM O .ENV (OU COM O NOME DA VARIÁVEL ERRADO) · SAÍDA REAL

```
Chave carregada? não
Traceback (most recent call last):
...
groq.GroqError: The api_key client option must be set either by
passing api_key to the client or by setting the GROQ_API_KEY
environment variable
```

O SDK cita GROQ_API_KEY porque é o nome padrão dele; no curso, confira GROQ_API no .env.

ModuleNotFoundError: No module named 'groq': o venv não está ativado, ou o pip install rodou fora dele.

CHAVE ERRADA OU APAGADA NO CONSOLE · SAÍDA REAL

```
Chave carregada? sim
Traceback (most recent call last):
...
groq.AuthenticationError: Error code: 401 - {'error': {'message':
'Invalid API Key', 'type': 'invalid_request_error', 'code':
'invalid_api_key'}}
```

Gerado com uma chave falsa de propósito. Copie a chave de novo ou crie outra.

Error code: 429: limite do plano estourado (aula 01). Espere os segundos indicados em `retry-after` .

Abrindo o *JupyterLab*

Na pasta `Eng_prompt`, com o `venv` ativado

```
jupyter lab
```

- ✓ O JupyterLab abre sozinho no navegador
- ✓ Rodando de dentro da pasta, ele já mostra os notebooks e enxerga o `.env`
- ✓ Para encerrar: `Ctrl` + `C` no terminal e confirme

Checklist do ambiente

- ☐ Python 3.10+
- ☐ Chave criada em console.groq.com/keys
- ☐ Pasta `Eng_prompt` com `.venv` ativado
- ☐ `pip install -r requirements.txt`
- ☐ `.env` com `GROQ_API=...` e `.gitignore`
- ☐ `teste_groq.py` respondeu

A interface do JupyterLab, as células e os atalhos ficam para a aula 03.

Teste o que *ficou*

1. Onde a chave da API deve ficar?

A) Escrita no notebook

B) No arquivo .env, fora do Git

C) No requirements.txt

D) Num print

B.

2. Qual o nome da variável usado nos notebooks?

A) GROQ_API

B) GROQ_API_KEY

C) API_KEY

D) OPENAI_API_KEY

A.

3. "No module named 'groq'" indica que:

A) A chave está errada

B) O venv não está ativado ou faltou instalar

C) O limite estourou

D) O modelo saiu do ar

B.

4. Erro 401 "Invalid API Key" se resolve:

A) Reinstalando o Python

B) Conferindo ou recriando a chave

C) Esperando um minuto

D) Mudando a temperatura

B.

Ambiente pronto, *chave protegida*

Chave criada no console, pasta com venv ativado, bibliotecas do `requirements.txt` instaladas, `GROQ_API` no `.env` (que fica fora do Git) e um teste que responde. A partir daqui, todo notebook começa com `load_dotenv()` e `Groq(api_key=...)`.

`console.groq.com/keys`

`.venv`

`pip install -r`

`.env + .gitignore`

`teste_groq.py`

Próxima aula → 03 · Introdução ao JupyterLab

Fontes: `console.groq.com/docs/quickstart`; `docs.python.org/3/library/venv.html` (ativação e `Set-ExecutionPolicy`); `jupyterlab.readthedocs.io` (Starting JupyterLab, versão 4.6.4); `pypi.org` (`groq 1.7.0`, `jupyterlab 4.6.4` e `python-dotenv 1.2.4` exigem Python 3.10+), consultados em outubro de 2026. Erros da seção 07 gerados com `exemplos/teste_groq.py` (sem `.env` e com uma chave falsa).